De la régression à la classification

TP 02 · Expériences guidées et exercices de programmation

Traduire les équations du cours en petits programmes NumPy lisibles. Distinguer ensuite l'apprentissage d'une probabilité, la décision et son évaluation.

> **À l'issue du TP :** effectuer un pas de gradient ; entraîner un classifieur logistique ; calculer les mesures de classification ; choisir un seuil sur la validation et l'évaluer sur un test réservé.

Vos fichiers

Fichier	Utilisation
`01_Demonstrations_FR.ipynb`	Exécuter les exemples, examiner les sorties et répondre aux questions d'observation.
`02_Exercices_FR.ipynb`	Compléter E1–E4 ; B1–B3 proposent des approfondissements.

Ouvrir un notebook

Colab : ouvrir colab.research.google.com → Fichier → Importer un notebook. Importer les démonstrations et se connecter à un environnement Python. Enregistrer/télécharger le travail modifié avant de quitter.

Jupyter local : utiliser un environnement Python existant, ou extraire l'archive et lancer ces commandes dans un terminal du dossier :

```
python -m pip install -r requirements.txt
python -m notebook
```

Ouvrir le notebook dans Jupyter. Exécuter dans l'ordre avec **Maj+Entrée**. Chaque notebook contient ses imports et ses données. Aucun GPU ni clé API n'est nécessaire. Une installation locale fonctionne hors ligne une fois les bibliothèques installées.

Lire le code

`np.array` crée un vecteur ; `*` multiplie terme à terme ; `np.mean` calcule une moyenne ; `np.sum(masque)` compte les valeurs vraies. Les indices commencent à 0.

`TODO` indique une fonction incomplète. `OK` indique la réussite des vérifications. En cas d'état incohérent, redémarrer le noyau/l'environnement et tout exécuter dans l'ordre.

D1 · Suivre un pas linéaire

Prédire la durée y (minutes) à partir de la distance x (kilomètres). Données du cours :

x	1	2	3
y	12	19	32

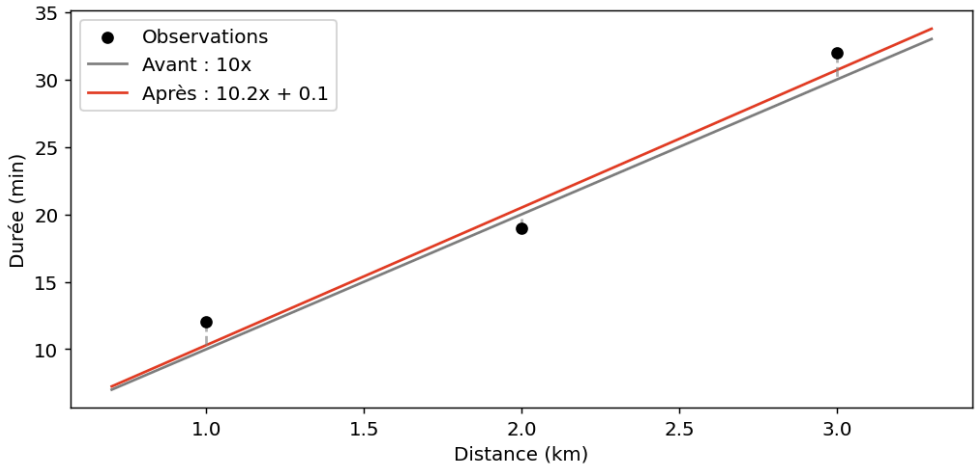
Prédiction → Résidus → Gradients moyens → Mise à jour

$$r_i = wx_i + b - y_i, \qquad J = \frac{1}{2n}\sum_i r_i^2$$

$$g_w = \frac{1}{n}\sum_i r_i x_i, \quad g_b = \frac{1}{n}\sum_i r_i, \quad (w, b) \leftarrow (w - \eta g_w, b - \eta g_b)$$

Partir de w=10, b=0 et η=0.1. Lire chaque ligne avant d'exécuter la cellule D1.

```
residual = w * x + b - y
dw = np.mean(residual * x)
db = np.mean(residual)
w_new, b_new = w - eta * dw, b - eta * db
```



Résultat attendu : r = [−2, 1, −2] ; (g_w, g_b) = (−2, −1). Le pas donne (10.2, 0.1). La perte passe de 1.5 à 1.138333.

Expliquer le changement

Les pointillés montrent les résidus initiaux. Un gradient négatif signifie qu'une petite augmentation du paramètre réduit localement la perte ; la mise à jour soustrait le gradient.

Question : identifier données, paramètres et pas d'apprentissage. Pourquoi les deux gradients doivent-ils utiliser les mêmes anciens paramètres ?

D2 · Apprendre une probabilité

La cible répond maintenant à une question : **la durée dépasse-t-elle 30 minutes ?** Poser c=1 si y>30, sinon c=0.

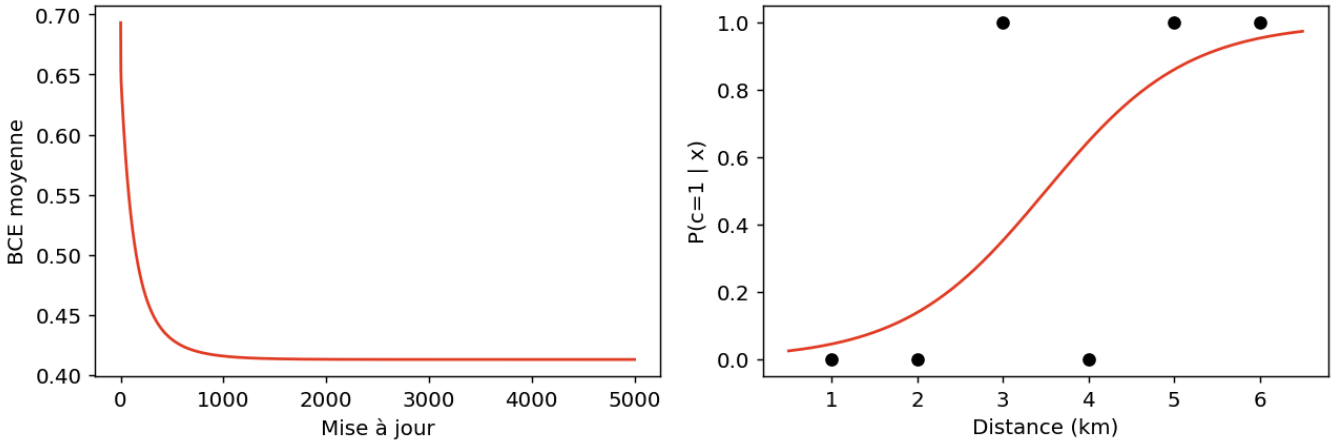
x	1	2	3	4	5	6
y	12	19	32	29	48	57
c	0	0	1	0	1	1

$$p_i = \sigma(wx_i + b), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

$$J = -\frac{1}{n}\sum_i [c_i \log p_i + (1-c_i)\log(1-p_i)]$$

$$g_w = \frac{1}{n}\sum_i (p_i - c_i)x_i, \qquad g_b = \frac{1}{n}\sum_i (p_i - c_i)$$

1. Exécuter le premier pas avec w=b=0 et η=0.1.
2. Vérifier : probabilités initiales 0.5, g_w=−7/12, g_b=0.
3. Exécuter 5 000 pas et examiner les deux graphiques.



> **Résultat attendu :** première perte 0.693147 → 0.665548. Après apprentissage : w≈1.213446, b≈−4.246823, BCE≈0.412998.

Relier code et théorie

`p−c` est la dérivée de sigmoïde + BCE par rapport au logit. Comme dans D1, moyenner les gradients sur tout le lot avant la mise à jour. Les logarithmes sont naturels.

La fonction fournie calcule l'expression stable équivalente `logaddexp(0,z)−c∗z`, évitant les débordements numériques des formules naïves avec exponentielles et logarithmes.

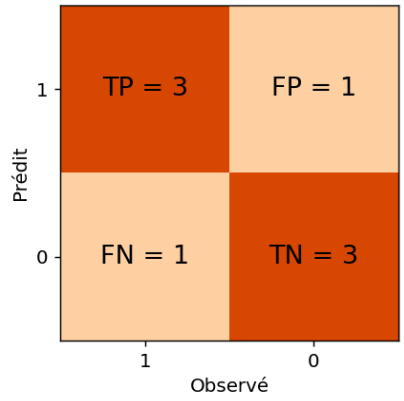
Questions : pourquoi des probabilités continues et des classes binaires ? Pourquoi la courbe ne passe-t-elle pas par chaque classe ? Qu'est-ce qui reste fixe lors d'une nouvelle prédiction ?

D3 · Évaluer une décision

Reprendre les huit scores de validation du cours. Cet exemple construit est indépendant des six observations d'entraînement de D2.

Cas	A	B	C	D	E	F	G	H
p	.95	.85	.70	.60	.45	.30	.20	.10
c	1	1	0	1	1	0	0	0

Prédire 1 si **p≥τ**, égalité comprise. Lire la matrice au seuil τ=0.5.



TP : positif prédit et observé.
FP : fausse alerte.
FN : positif manqué.
TN : négatif correctement rejeté.

Dans le cours, lignes = **prédictions**, colonnes = **observations**, ordre [1,0]. Scikit-learn place les observations en lignes : utiliser `confusion_matrix(..., labels=[1,0]).T` pour retrouver cette disposition.

Mesure	Calcul	Interprétation
Exactitude	(TP+TN)/n	Quelle proportion est correcte ?
Précision	TP/(TP+FP)	Parmi les positifs prédits, combien sont vrais ?
Rappel / TPR	TP/(TP+FN)	Parmi les positifs observés, combien sont détectés ?
F1	2TP/(2TP+FP+FN)	Équilibre harmonique précision–rappel.
Spécificité	TN/(TN+FP)	Parmi les négatifs, combien sont rejetés ?

Exécuter D3 à τ=0.5 et 0.8. À 0.5 : TP/FP/FN/TN=3/1/1/3, les cinq mesures valent 0.75. À 0.8 : 2/0/2/4, précision=1, rappel=0.5, F1=2/3.

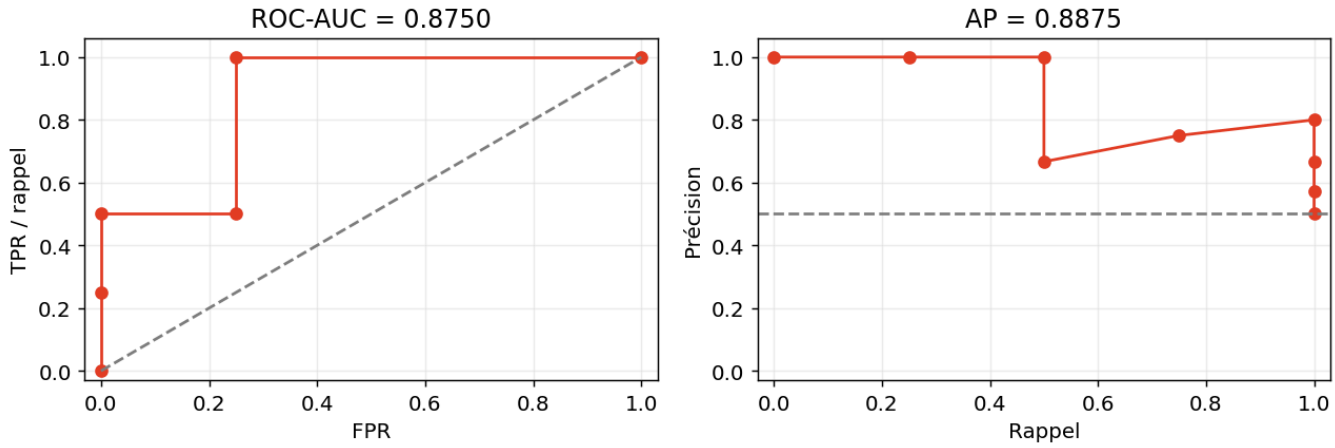
> **Interpréter :** l'exactitude reste à 0.75. Quelle fausse alerte disparaît ? Quel positif supplémentaire est manqué ? Choisir la mesure selon l'erreur qui compte dans l'application.

Un dénominateur nul rend le ratio indéfini. Le TP utilise 0.0 par convention ; cette convention ne démontre pas l'utilité du classifieur.

D3 · Lire ROC et précision–rappel

Un seuil produit une règle de décision. Faire varier ce seuil produit une courbe. Fournir les **scores d'origine** aux fonctions ROC et PR.

$$\mathrm{TPR} = \frac{TP}{TP+FN}, \qquad \mathrm{FPR} = \frac{FP}{FP+TN}$$



ROC : classement à travers les seuils

ROC trace TPR en fonction de FPR. Ici AUC=0.875 : le score positif gagne 14 des 16 paires positif–négatif. Une égalité compte pour une demi-victoire.

La diagonale correspond au classement aléatoire en espérance. AUC mesure le classement ; elle ne choisit pas le seuil à déployer et ne mesure pas la calibration des probabilités.

PR : détection et fausses alertes

PR trace la précision en fonction du rappel. La prévalence positive vaut 4/8=0.5, représentée comme niveau de référence. PR est particulièrement informative lorsque les positifs sont rares.

$$\mathrm{AP} = \sum_k (R_k - R_{k-1}) P_k$$

Dans cette formule, le rappel augmente lorsque le seuil diminue. Ici, les rangs des positifs sont 1, 2, 4 et 5 :

$$\mathrm{AP} = \tfrac{1}{4}\left(1 + 1 + \tfrac{3}{4} + \tfrac{4}{5}\right) = 0.8875$$

`average_precision_score` pondère la précision par les gains de rappel. **Ne pas remplacer AP par l'aire PR calculée par trapèzes.** Les segments du graphique relient les points ; ils ne définissent pas le calcul d'AP.

> **Questions :** passer de τ=0.5 à τ=0.8 change-t-il AUC ou AP si les scores restent fixes ? Une AUC élevée garantit-elle un rappel élevé à tous les seuils ?

E1–E2 · Programmer l'apprentissage

Ouvrir `02_Exercices_FR.ipynb` et exécuter les imports. Compléter les fonctions indiquées sans modifier les vérifications.

E1 · Un pas linéaire réutilisable

```
linear_step(x, y, w, b, eta) → (w_new, b_new)
```

x et y sont des tableaux NumPy 1D non vides de même longueur. Utiliser la convention de D1 : J=MSE/2.

1. Calculer les prédictions et les résidus.
2. Calculer les deux gradients moyens avec les anciens paramètres.
3. Renvoyer les deux paramètres mis à jour simultanément.
4. Exécuter les vérifications. Le deuxième pas part de (10.2, 0.1), avec η=0.1, et donne environ (10.286667, 0.15).

> **Réponse écrite :** expliquer pourquoi modifier w avant le calcul du gradient de b change l'algorithme.

E2 · Une fonction d'apprentissage logistique

```
fit_logistic(x, c, eta=0.1, updates=5000)
→ {"w": ..., "b": ..., "losses": ...}
```

Tableaux 1D non vides de même longueur ; c contient les classes 0/1. Initialiser w=b=0. Utiliser `sigmoid` et `mean_bce`, déjà fournies.

1. Enregistrer la perte avant toute mise à jour.
2. À chaque pas : prédire p, calculer les deux gradients moyens, mettre à jour w et b, puis enregistrer la nouvelle perte.
3. Renvoyer les paramètres finaux et l'historique complet.
4. Exécuter les vérifications et examiner la courbe de perte.

Vérification	Résultat attendu
Longueur de l'historique	`updates + 1`
Zéro mise à jour	w=b=0 et une perte initiale
Données du cours après 5 000 pas	BCE ≈ 0.412998

> **Réponse écrite :** relier p−c à la dérivée de BCE. Que démontre une perte d'entraînement décroissante ? Que faut-il examiner pour évaluer de nouvelles données ?

E3–E4 · Mesurer et valider

E3 · Votre bilan de classification

```
classification_report_manual(y_true, probabilities, tau)
→ {"TP": ..., "FP": ..., "FN": ..., "TN": ...,
   "accuracy": ..., "precision": ..., "recall": ...,
   "f1": ..., "specificity": ...}
```

Tableaux NumPy 1D non vides de même longueur et classes binaires. Prédire 1 avec $p \geq \tau$. Compter les quatre issues par masques booléens, puis utiliser les formules page 4. Effectifs entiers ; renvoyer **0.0 si un dénominateur est nul**.

Exécuter les vérifications aux seuils 0.5 et 0.8, au cas d'égalité et en l'absence de positifs. Ne pas appeler de fonction de mesure d'une bibliothèque dans votre implémentation.

> **Réponse écrite :** identifier les populations aux dénominateurs de la précision et du rappel. Comment une même exactitude peut-elle masquer des erreurs différentes ?

E4 · Un protocole d'évaluation complet

Les données synthétiques simulent une durée de marche supérieure à 30 minutes. Une graine fixe produit 200 cas d'entraînement, 100 de validation et 100 de test. Le code crée les données ; les CSV sont facultatifs.

Entraîner w,b → Valider τ → Figer → Tester

```
choose_threshold(y_val, p_val, candidates) → tau
```

1. Exécuter les données. Ajuster w,b avec E2 sur **l'entraînement seul**.
2. Dans `choose_threshold`, utiliser E3 pour calculer F1 de validation aux seuils [0.2, 0.35, 0.5, 0.65, 0.8].
3. Renvoyer le seuil maximisant F1. En cas d'égalité, choisir le **seuil le plus élevé**.
4. Examiner la table de validation. Figer (w,b,τ).
5. Exécuter le test. Rapporter les effectifs de confusion, les cinq mesures, ROC–AUC et AP. Ne pas réajuster après leur lecture.

> **Réponse écrite :** expliquer les rôles de l'entraînement, de la validation et du test. Le meilleur seuil de validation donne-t-il nécessairement le meilleur F1 de test ? Pourquoi ?

Distinguer les choix : la descente de gradient ajuste les probabilités en minimisant BCE d'entraînement. Le seuil définit une décision en maximisant F1 de validation. Changer τ ne réentraîne pas w,b.

Approfondissements et travail à rendre

B1 · Comparer les pas d'apprentissage

Réutiliser E1 sur les trois cas de régression. Pour chaque η parmi [0.01, 0.1, 0.4], partir de (w,b)=(10,0) et effectuer 30 pas. Tracer les trois pertes avec un axe vertical logarithmique.

Expliquer : pourquoi garder données et départ identiques ? Quel pas progresse lentement ? Lequel diverge ? Une perte décroissante avec un pas ne garantit pas la stabilité de tous les autres.

B2 · Construire ROC vous-même

```
roc_manual(y_true, scores) → (fpr, tpr)
```

Les deux classes sont présentes. Partir au-dessus des scores, puis parcourir les **scores distincts dans l'ordre décroissant**. Regrouper les scores égaux. Renvoyer les tableaux contenant (0,0) et (1,1).

Utiliser les huit cas de D3. Calculer l'aire par trapèzes et retrouver AUC=0.875. Tester une paire positif–négatif ex æquo : AUC doit valoir 0.5.

B3 · Interroger une forte exactitude

Construire 1 000 classes : 10 positives et 990 négatives. Attribuer la probabilité 0 à chaque cas, avec τ=0.5. Utiliser E3 pour calculer exactitude, rappel et F1. Expliquer si ce classifieur détecte des positifs.

Rendre votre travail

1. Enregistrer le notebook complété sous `Nom_Prenom_TP02.ipynb`.
2. Inclure code E1–E4, sorties, explications et approfondissements réalisés. Conserver le seuil choisi et le bilan final de test.
3. Redémarrer le noyau/l'environnement et tout exécuter avant de rendre. La réussite des vérifications ne remplace pas l'explication.

> **Auto-évaluation :** distinguez-vous probabilité et classe, optimisation et généralisation, mesure à un seuil et mesure de classement ?

Pour approfondir

- Stanford CS229 · Apprentissage supervisé : modèles linéaires et logistiques.
- Cornell CS4780 · Gradient et régression logistique.
- scikit-learn · Matrice de confusion : convention lignes/colonnes.
- scikit-learn · Précision–rappel : courbe et précision moyenne.
- Jupyter · Installation.